

COURSE INFORMATION

Description:

Learners are introduced to algorithms as structured approaches to solving problems. The course focuses on analyzing problems, selecting appropriate strategies, and implementing solutions using common data structures. Students explore how different approaches affect performance through basic analysis techniques and comparison of solutions. Emphasis is placed on reasoning, debugging, and refining solutions both manually and with AI-assisted tools. By the end of the course, learners are able to evaluate, adapt, and explain algorithmic approaches used to solve problems effectively

Career Cluster: Information Technology

Instructional Level: A.A.S. - Associate in Applied Science

Total Credits: 3

Total Hours: 72

PRE/COREQUISITES

- Prerequisite: 10-152-117 Python Programming

TEXTBOOKS

50 Algorithms Every Programmer Should Know: Tackle computer science challenges with classic to modern algorithms in machine learning, software design, data systems, and cryptography - 2nd ed. Edition - Imran Ahmadad

CORE ABILITIES

1. Act Professionally

Status: Active

Criteria

- 1.1. Take responsibility for one's own actions.
- 1.2. Conform to the technical standards of a profession.
- 1.3. Practice morally responsible behavior.
- 1.4. Take responsibility for staying up-to-date
- 1.5. Maintain confidentiality.
- 1.6. Exhibit respect for people and property.
- 1.7. Exhibit a sense of global awareness.
- 1.8. Display appearance appropriate for work environment.

2. Communicate Clearly

Status: Active

Criteria

- 2.1. Speak clearly so others can understand.
- 2.2. Write with clarity so that others can understand.
- 2.3. Select appropriate means to convey a message.
- 2.4. Ask questions for clarification.
- 2.5. Participate in discussions and group presentations.
- 2.6. Interpret nonverbal communications.
- 2.7. Use active listening skills.
- 2.8. Apply standards of spelling, English, grammar, and punctuation.

3. Solve Problems

Status: Active

Criteria

- 3.1. Use critical, creative thinking skills.
- 3.2. Apply problem solving steps.
- 3.3. Demonstrate open-mindedness.
- 3.4. Evaluate alternatives to choosing a solution.
- 3.5. Implement solutions appropriately.
- 3.6. Use research to solve problems.
- 3.7. Use appropriate mathematical calculations.

4. Value Learning

Status: Active

Criteria

- 4.1. Assume responsibility for lifelong learning.
- 4.2. Identify own learning needs.
- 4.3. Apply effective learning processes.
- 4.4. Help other to learn effectively.

5. Work Cooperatively

Status: Active

Criteria

- 5.1. Demonstrate respect for self and others.
- 5.2. Contribute to a group activity with ideas, suggestions, and effort.
- 5.3. Complete own share of tasks necessary to finish a group project.
- 5.4. Maintain a safe and healthy work environment for self/group.
- 5.5. Set goals, standards, limits, for self/group.
- 5.6. Display effective interpersonal skills.
- 5.7. Resolve conflicts in a constructive manner.
- 5.8. Seek help when needed.
- 5.9. Demonstrate ability to work with a diverse population.

6. Work Productively

Status: Active

Criteria

- 6.1. Attend regularly and on time.
- 6.2. Exhibit organizational skills.
- 6.3. Locate resources for problem solving.
- 6.4. Display productive work ethic.
- 6.5. Maintain necessary knowledge and skills.
- 6.6. Use effective, efficient processes.
- 6.7. Show self-direction in starting tasks.
- 6.8. Demonstrate reliability.
- 6.9. Follow directions.
- 6.10. Complete required tasks on time.

COURSE COMPETENCIES

1. Frame problems and analyze algorithmic requirements

Domain: Cognitive Level: Analyzing Status: WIP

Assessment Strategies

- 1.1. Problem-framing activities
- 1.2. Lab-based algorithm assignments
- 1.3. Pseudocode, diagram, trace, or explanation artifacts
- 1.4. Project assignments or quiz

Criteria

- 1.1. Decompose problems into clear algorithmic steps
- 1.2. Identify inputs, outputs, constraints, assumptions, and edge cases
- 1.3. Distinguish between clear and ambiguous problem statements
- 1.4. Represent solution logic using pseudocode, diagrams, traces, or code
- 1.5. Explain why a proposed approach fits the problem

Learning Objectives

- 1.a. Define an algorithm and its key characteristics
- 1.b. Explain problem decomposition and abstraction
- 1.c. Identify problem constraints, assumptions, and edge cases
- 1.d. Describe algorithmic solutions before implementation

2. Select data structures and representations for algorithmic solutions

Domain: Cognitive Level: Analyzing Status: WIP

Assessment Strategies

- 2.1. Data-structure selection activities
- 2.2. Representation comparison tasks
- 2.3. Lab-based algorithm assignments
- 2.4. Project assignments or quiz

Criteria

- 2.1. Use lists, dictionaries, sets, stacks, queues, trees, and graphs at an introductory level
- 2.2. Match data structures to access, lookup, ordering, traversal, or membership needs

- 2.3. Explain how representation affects solution clarity and performance
- 2.4. Represent simple relationships, workflows, or traversals using appropriate structures
- 2.5. Justify representation choices using task requirements and constraints

Learning Objectives

- 2.a. Identify common data structures used in algorithmic solutions
- 2.b. Explain basic tradeoffs among common data structures
- 2.c. Describe how representation affects algorithm design
- 2.d. Use Python structures to support algorithm implementation

3. Implement and test introductory algorithms using Python

Domain: Cognitive Level: Applying Status: WIP

Assessment Strategies

- 3.1. Python algorithm implementation labs
- 3.2. Search, sort, recursion, and traversal activities
- 3.3. Testing and expected-outcome checks
- 3.4. Project assignments or quiz

Criteria

- 3.1. Implement introductory searching, sorting, recursive, iterative, and graph traversal algorithms
- 3.2. Translate algorithmic logic into readable Python code
- 3.3. Test solutions using normal cases, edge cases, and expected outcomes
- 3.4. Identify and correct logical errors
- 3.5. Use libraries appropriately when implementation is not the learning target

Learning Objectives

- 3.a. Describe how core programming constructs support algorithms
- 3.b. Implement algorithmic steps in Python
- 3.c. Explain the relationship between pseudocode and executable code
- 3.d. Apply testing to verify algorithm correctness

4. Evaluate correctness, efficiency, and tradeoffs among algorithmic approaches

Domain: Cognitive Level: Evaluating Status: WIP

Assessment Strategies

- 4.1. Correctness and efficiency comparisons
- 4.2. Big-O reasoning activities
- 4.3. Timing or trace evidence tasks
- 4.4. Project assignments or quiz

Criteria

- 4.1. Verify whether an algorithm produces expected results
- 4.2. Compare multiple solutions to the same problem
- 4.3. Apply introductory Big-O reasoning to describe growth behavior
- 4.4. Discuss tradeoffs among correctness, simplicity, readability, efficiency, and maintainability
- 4.5. Use timing data, traces, diagrams, or other evidence to support evaluation

Learning Objectives

- 4.a. Define correctness in an algorithmic context
- 4.b. Explain common growth patterns such as constant, linear, quadratic, and logarithmic behavior
- 4.c. Identify common sources of inefficiency or incorrect assumptions
- 4.d. Describe tradeoffs in algorithm selection

5. Use observable evidence to explain algorithm behavior

Domain: Cognitive Level: Applying Status: WIP

Assessment Strategies

- 5.1. Trace, table, chart, or diagram activities
- 5.2. Visual or tangible algorithm demonstrations
- 5.3. Comparative explanation tasks
- 5.4. Project assignments or quiz

Criteria

- 5.1. Use traces, tables, charts, diagrams, simulations, or other evidence to show algorithm behavior
- 5.2. Connect evidence to claims about correctness, efficiency, or tradeoffs
- 5.3. Represent search, sort, traversal, similarity, ranking, or clustering behavior when appropriate
- 5.4. Explain what the evidence reveals about an algorithm's operation
- 5.5. Use evidence to communicate algorithmic behavior to technical or non-technical audiences

Learning Objectives

- 5.a. Identify forms of observable evidence for algorithm behavior
- 5.b. Explain how traces, timing, diagrams, or tables support algorithm analysis
- 5.c. Describe how evidence strengthens algorithmic explanation
- 5.d. Connect visual or tangible evidence to correctness and tradeoff claims

6. Connect algorithmic thinking to AI, data systems, and responsible tool use

Domain: Cognitive Level: Evaluating Status: WIP

Assessment Strategies

- 6.1. AI/data bridge activities
- 6.2. Similarity, clustering, recommendation, hashing, or representation tasks
- 6.3. AI/tool-use justification activities
- 6.4. Project assignments or quiz

Criteria

- 6.1. Recognize how similarity, clustering, recommendation, hashing, and representation support AI and data systems
- 6.2. Explain how assumptions, bias, incomplete data, or poor representation can affect algorithmic outcomes
- 6.3. Compare manual, library-supported, and AI-assisted solutions when appropriate
- 6.4. Identify limitations, inaccuracies, or hidden assumptions in AI-generated algorithmic output
- 6.5. Use AI or other tools transparently while validating output and explaining human decisions

Learning Objectives

- 6.a. Describe selected algorithmic foundations of AI and data systems
- 6.b. Explain why algorithms require validation and interpretation

- 6.c. Identify responsible uses and limits of AI-assisted development
- 6.d. Communicate tool-assisted solution choices and tradeoffs clearly

7. Solve algorithmic problems using structured strategies

Domain: Cognitive Level: Analyzing Status: WIP

Assessment Strategies

- 7.1. Troubleshooting activities
- 7.2. Strategy-selection tasks
- 7.3. Lab-based algorithm assignments
- 7.4. Project assignments or quiz

Criteria

- 7.1. Analyze problems before selecting a solution strategy
- 7.2. Test assumptions and revise approaches when evidence shows a mismatch
- 7.3. Recover from incorrect logic, failed tests, or inefficient approaches
- 7.4. Use resources, examples, documentation, peers, or targeted support appropriately
- 7.5. Persist through algorithmic uncertainty without bypassing reasoning

Learning Objectives

- 7.a. Describe structured problem-solving strategies for algorithmic work
- 7.b. Explain how assumption testing supports better solutions
- 7.c. Identify productive recovery strategies when an algorithm does not work

8. Communicate algorithmic choices, evidence, and tradeoffs clearly

Domain: Cognitive Level: Applying Status: WIP

Assessment Strategies

- 8.1. Algorithm explanation activities
- 8.2. Pseudocode, diagram, or evidence discussion
- 8.3. Presentation or walkthrough tasks
- 8.4. Project assignments or quiz

Criteria

- 8.1. Explain algorithmic choices using clear technical language
- 8.2. Describe assumptions, constraints, evidence, and tradeoffs
- 8.3. Use pseudocode, diagrams, traces, tables, or examples to support explanation
- 8.4. Communicate algorithmic reasoning to technical or non-technical audiences
- 8.5. Document AI or tool use when it affects the solution process

Learning Objectives

- 8.a. Explain the role of communication in algorithmic problem solving
- 8.b. Identify ways to describe algorithm behavior and tradeoffs
- 8.c. Describe how evidence supports clear technical explanation

9. Work productively through algorithmic labs, checkpoints, and revisions

Domain: Affective Level: Responding Status: WIP

Assessment Strategies

- 9.1. Lab-based algorithm assignments
- 9.2. Project checkpoints
- 9.3. Organized submission artifacts
- 9.4. Project assignments or quiz

Criteria

- 9.1. Use lab time to make visible progress on algorithmic tasks
- 9.2. Organize files, notes, traces, data, diagrams, and submissions
- 9.3. Follow assignment requirements and checkpoint expectations
- 9.4. Complete required work by stated deadlines
- 9.5. Demonstrate self-direction when framing, implementing, testing, or explaining solutions

Learning Objectives

- 9.a. Identify productive work habits for algorithmic labs
- 9.b. Explain how organization supports testing and explanation
- 9.c. Describe how incremental progress supports stronger algorithmic work

10. Apply feedback, references, and comparison to deepen algorithmic learning

Domain: Affective Level: Valuing Status: WIP

Assessment Strategies

- 10.1. Revised lab or project submissions
- 10.2. Reflection or comparison activities
- 10.3. Textbook/reference-use activities
- 10.4. Project assignments or quiz

Criteria

- 10.1. Use instructor feedback to improve algorithmic reasoning, code, or explanation
- 10.2. Use textbook, reference, documentation, or example materials appropriately
- 10.3. Compare approaches to deepen understanding rather than only finish a task
- 10.4. Revise code, evidence, or explanations based on testing and feedback
- 10.5. Ask useful questions that reveal or resolve algorithmic misunderstandings

Learning Objectives

- 10.a. Explain how feedback supports algorithmic learning
- 10.b. Identify references that support algorithmic reasoning and implementation
- 10.c. Describe how comparison and revision improve understanding

11. Collaborate respectfully in algorithmic reasoning and technical review

Domain: Affective Level: Responding Status: WIP

Assessment Strategies

- 11.1. Peer explanation or walkthrough activities
- 11.2. Partner testing or review tasks
- 11.3. Collaborative problem-solving activities
- 11.4. Project assignments or quiz

Criteria

- 11.1. Participate respectfully in shared algorithmic problem solving
- 11.2. Explain reasoning, tests, or evidence to peers
- 11.3. Give and receive useful feedback on logic, tradeoffs, or clarity
- 11.4. Share responsibility during partner or group activities
- 11.5. Support peers without replacing their reasoning or work

Learning Objectives

- 11.a. Describe expectations for respectful technical collaboration
- 11.b. Explain how peer review can improve algorithmic reasoning
- 11.c. Identify appropriate boundaries for collaboration in algorithmic work

12. Act professionally in algorithmic coursework and tool-supported work

Domain: Affective Level: Valuing Status: WIP

Assessment Strategies

- 12.1. Lab participation
- 12.2. Project or explanation submissions
- 12.3. AI/tool-use documentation
- 12.4. Project assignments or quiz

Criteria

- 12.1. Arrive prepared for lab, discussion, testing, and explanation activities
- 12.2. Take responsibility for their work, conduct, and tool use
- 12.3. Follow course, assignment, and academic integrity expectations
- 12.4. Use AI, libraries, references, and external examples transparently and ethically
- 12.5. Demonstrate accountability when explaining assumptions, limitations, and decisions

Learning Objectives

- 12.a. Explain professional expectations in algorithmic coursework
- 12.b. Identify ethical considerations in tool-supported algorithmic work
- 12.c. Describe how transparency and accountability support trust in technical decisions